AI WEATHERWISE API

(React Application)



Academic Year: 2026–2027

GOVERNMENT ARTS AND SCIENCE,COLLEGE

WIMCO NAGAR

THIRUVOTTRIYUR,CHENNAI-19

Team ID : SWTID-2026-4867
AI WEATHERWISE API
Team Size : 5

Team Leader : DHARANYA A

Team member : DHARSHINI M

Team member : VINODHA S

Team member : Gnanapriya V

Team member : HEMA J

index

S. No.	CONTENTS
1	Introduction
2	Problem Statement
3	Objectives
4	Scope of the Project
5	Technologies Used
6	System Requirements
7	System Workflow
8	System Architecture
9	Functional Requirements
10	Non-Functional Requirements
11	Modules of the Project
12	API Request and Response
13	Implementation
14	Screenshots
15	Advantages
16	Limitations
17	Future Enhancement
18	Applications
19	Conclusion

1. INTRODUCTION:

Weather plays an important role in our daily life. Information about temperature, humidity, wind, rainfall, and other weather conditions is useful for travel, outdoor activities, agriculture, transportation, and many other applications.

AI WeatherWise API is an API-based project developed to provide weather-related information in a simple and accessible way. The system uses API technology to receive requests and provide the required weather information in a structured format.

The project demonstrates how Application Programming Interfaces (APIs) can be used to connect applications with weather data services. It helps users or other applications request weather information without manually collecting the required data.

The project also provides practical knowledge of API communication, request processing, data handling, and response generation. It can serve as a foundation for developing more advanced weather applications with features such as weather forecasting, location-based services, weather alerts, and AI-based analysis.

Overall, AI WeatherWise API demonstrates the practical use of modern API technology for accessing and delivering weather information efficiently.

2. PROBLEM STATEMENT:

Weather information is required for various daily activities such as travelling, outdoor planning, agriculture, transportation, and other weather-dependent activities. However,

obtaining and integrating weather information into applications can be difficult without a proper system.

Traditional methods of accessing weather information may require users to search through different sources manually. This can make the process time-consuming and less convenient.

Therefore, there is a need for a simple and efficient API-based system that can provide weather-related information in a structured and accessible format.

The AI WeatherWise API project addresses this problem by providing an API-based approach for requesting, processing, and delivering weather information. The system can also serve as a foundation for integrating weather data into web and other applications.

3. OBJECTIVES:

The main objectives of the AI WeatherWise API project are:

1. To develop an API-based system for providing weather-related information.
2. To provide weather information in a simple and structured format.
3. To understand the basic concepts of API communication and integration.
4. To process user requests and provide appropriate weather information.
5. To demonstrate how weather data can be accessed through an API.
6. To improve practical knowledge of web-based application development.

7. To provide a system that can be integrated with other applications.
8. To create a foundation for future development of advanced weather-based applications.

4. SCOPE OF THE PROJECT:

The AI WeatherWise API project focuses on providing weather-related information through an API-based system. The project can be used as a foundation for applications that require weather data.

The scope of the project includes:

- Providing weather information through an API.
- Processing requests related to weather data.
- Returning weather information in a structured format.
- Supporting integration with web and other applications.
- Making weather data easier to access and use.
- Providing a basic platform for developing advanced weather applications.
- Supporting future integration of forecasting and alert features.
- Providing opportunities for AI-based weather analysis and prediction.

The project can be further expanded to include location-based weather services, weather forecasts, notifications, graphical dashboards, and mobile application integration.

5. TECHNOLOGIES USED:

The AI WeatherWise API project uses web and API-based technologies to collect, process, and provide weather-related information.

5.1 API

An Application Programming Interface (API) is used to enable communication between the application and the weather data service. It allows the application to send requests and receive weather information.

5.2 JSON

JavaScript Object Notation (JSON) is used to represent and exchange structured data between the API and the application. It provides weather information in an easy-to-read format.

5.3 Web Technologies

Web-based technologies are used to create and interact with the API. They support communication between the user or application and the weather service.

5.4 Weather Data Service

A weather data service provides the required weather information such as temperature, humidity, weather condition, wind speed, and other available parameters.

5.5 API Testing Tool

An API testing tool or web browser can be used to send requests to the API and verify the responses received from the system.

5.6 Development Environment

A suitable code editor and development environment are used to develop, modify, and manage the project.

Note: The exact technologies and tools used may vary depending on the implementation of the project.

6. SYSTEM REQUIREMENTS:

The system requirements specify the hardware and software resources required to develop and use the AI WeatherWise API project.

6.1 Hardware Requirements

The minimum hardware requirements for the project are:

- Processor: Intel Core i3 or equivalent
- RAM: 4 GB or above
- Storage: At least 10 GB of free space
- Display: Standard monitor or laptop display
- Input Devices: Keyboard and mouse/touchpad
- Internet: Stable internet connection

6.2 Software Requirements

The software requirements for the project include:

- Operating System: Windows / Linux / macOS
- Web Browser: Google Chrome, Microsoft Edge, or equivalent
- Code Editor: Visual Studio Code or any suitable code editor
- API Testing: Browser or API testing tool
- Internet Connection: Required for accessing weather data services
- API Service: Weather API used by the project

6.3 User Requirements

The user should have:

- Basic knowledge of using a web browser.
- Basic understanding of weather information.
- Internet connectivity for accessing the API.
- Required API access, if authentication is used.

These requirements provide the basic environment needed to develop, run, and access the AI WeatherWise API project.

7. SYSTEM WORKFLOW:

The AI WeatherWise API follows a simple workflow to receive a weather request, process the request, obtain the required weather information, and return the response to the user or application.

7.1 Workflow

User / Client

↓

Enter Location / Send Request

↓

API Request

↓

Request Processing

↓

Weather Data Retrieval

↓

Data Processing

↓

Generate Response

↓

Display Weather Information

7.2 Workflow Explanation

Step 1 – User Request

The user or client application provides the required location or weather-related input.

Step 2 – API Request

The application sends the request to the AI WeatherWise API.

Step 3 – Request Processing

The API receives the request and processes the provided information.

Step 4 – Weather Data Retrieval

The system obtains the required weather information from the configured weather data service.

Step 5 – Data Processing

The retrieved weather data is processed and arranged into a structured format.

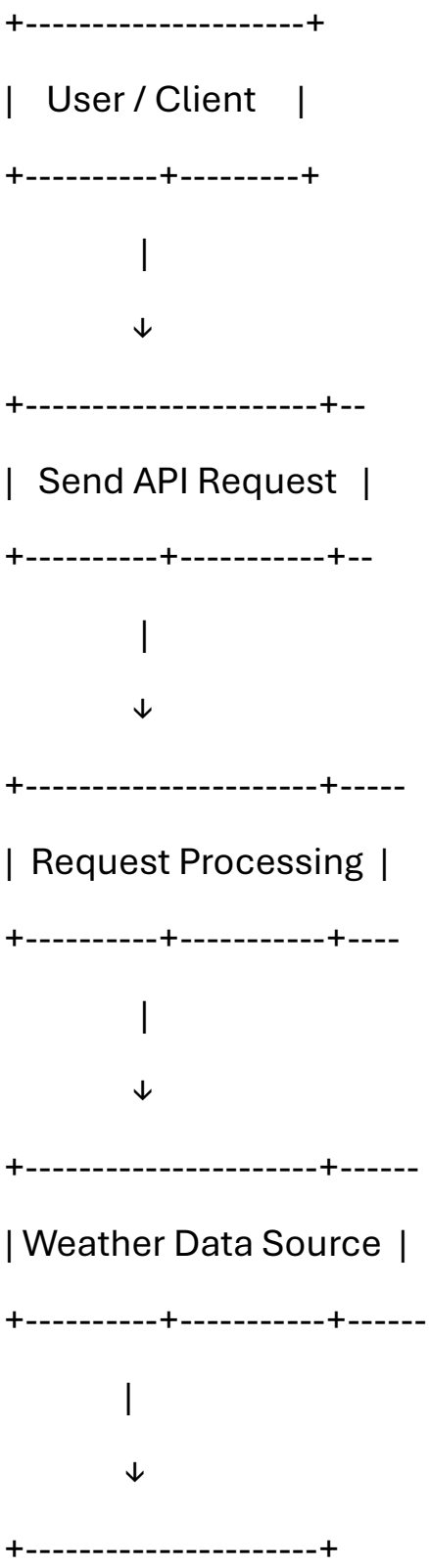
Step 6 – Response Generation

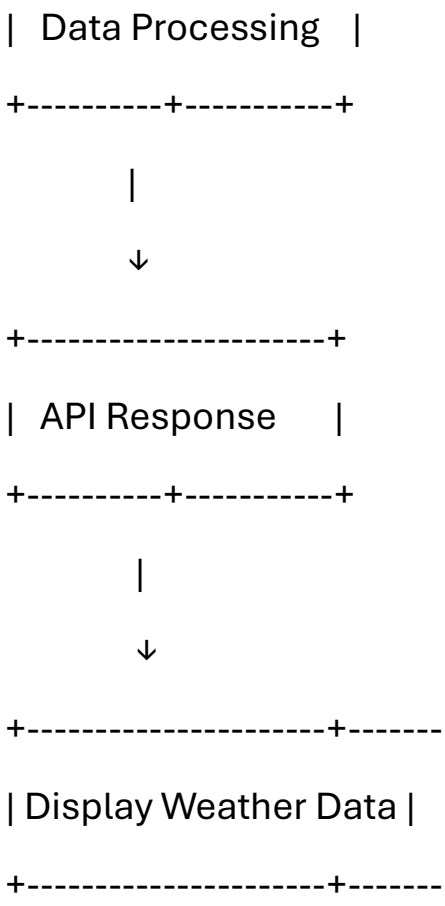
The API generates a response containing the requested weather information.

Step 7 – Display Information

The final response is returned to the user or application and displayed in an understandable format.

7.3 Basic Workflow Diagram





8. SYSTEM ARCHITECTURE:

The AI WeatherWise API follows a simple API-based architecture in which the client communicates with the weather service through the API layer.

The architecture consists of different components that work together to process the user's request and provide the required weather information.

8.1 Main Components

1. Client Layer

The client can be a user, web application, or other application that sends a request to obtain weather information.

2. API Layer

The API layer receives the request from the client and handles the communication between the client and the weather data service.

3. Request Processing Layer

This layer validates and processes the information provided in the API request.

4. Weather Data Source

The weather data source provides the required information such as temperature, humidity, weather condition, wind speed, and other available details.

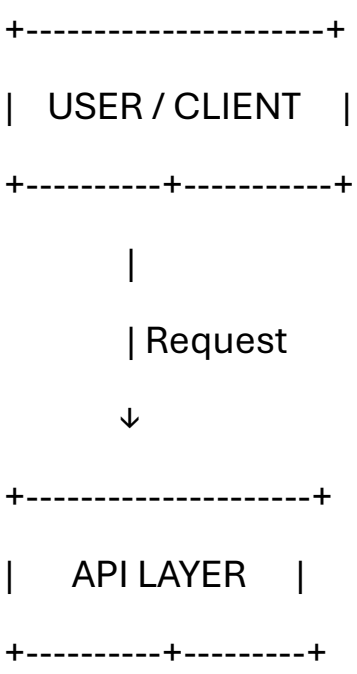
5. Response Processing Layer

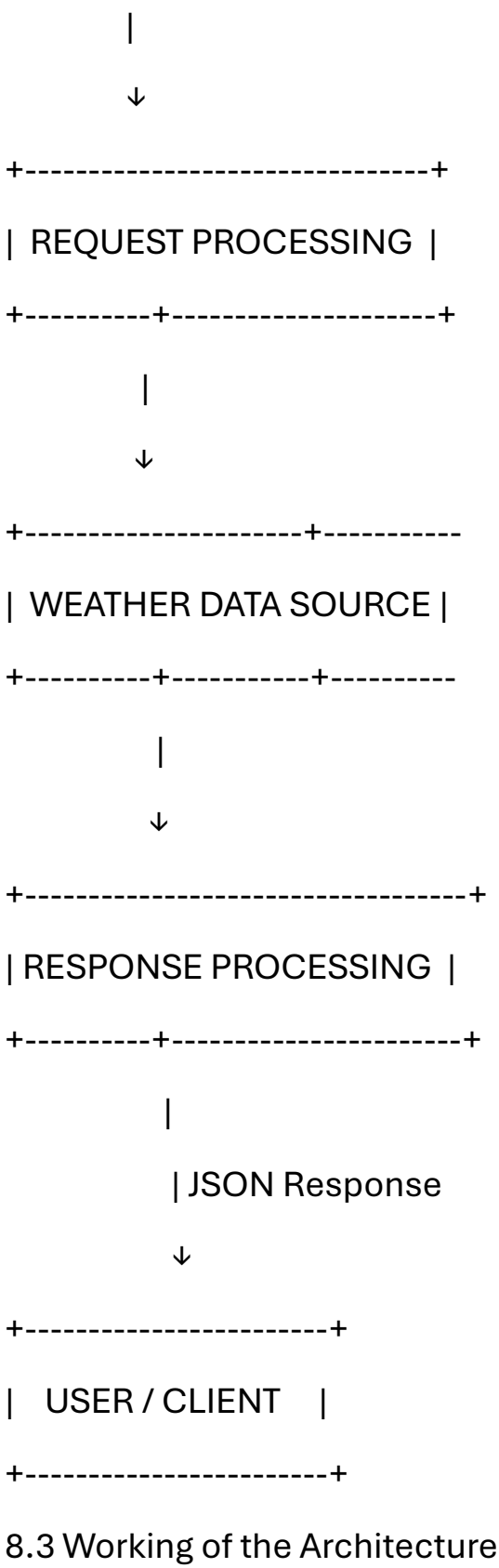
The retrieved weather information is processed and arranged into a structured response, generally using JSON format.

6. Response Layer

The final response is sent back to the client, where the weather information can be displayed or used by another application.

8.2 Architecture Flow





8.3 Working of the Architecture

The client first sends a weather-related request to the API. The API receives and processes the request, then communicates with the configured weather data source.

After receiving the required information, the system processes the data and generates a structured response. Finally, the response is returned to the client for display or further use.

This architecture provides a simple and flexible approach for integrating weather information into different applications.

9. FUNCTIONAL REQUIREMENTS:

The functional requirements describe the main operations and services that the AI WeatherWise API is expected to provide. These requirements explain how the system receives weather requests, processes the information, and provides a structured response to the user or application.

9.1 Accept Weather Request

The system should accept a request from the user or client application for weather information based on the required location or parameters.

9.2 Validate Input

The system should check whether the required input is provided correctly. Invalid or incomplete inputs should be handled appropriately.

9.3 Process API Request

The API should receive and process the incoming request and identify the information required to generate the weather response.

9.4 Retrieve Weather Data

The system should obtain the required weather information from the configured weather data source or service.

9.5 Process Weather Information

The retrieved weather data should be processed into a clear and structured format that can be easily understood by the client application.

9.6 Generate API Response

The system should generate a structured response containing relevant weather information such as:

- Location
- Temperature
- Weather condition
- Humidity
- Wind information, where available

9.7 Display Weather Information

The client application should be able to receive the API response and display the weather information to the user.

9.8 Handle Invalid Requests

The system should provide an appropriate error response when the request contains missing, invalid, or unsupported information.

9.9 Support Application Integration

The API should allow other applications or client interfaces to request and use the available weather information.

10. NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements describe the quality, performance, security, and usability characteristics of the AI WeatherWise API. These requirements ensure that the system works efficiently and provides a reliable experience for users and applications.

10.1 Performance

The API should process requests efficiently and provide weather information within a reasonable response time.

10.2 Reliability

The system should provide consistent responses when valid requests are submitted and should handle errors properly.

10.3 Usability

The API response should be simple, clear, and easy for users or other applications to understand and use.

10.4 Scalability

The system should be designed so that it can support an increasing number of users and API requests when required.

10.5 Security

The API should protect sensitive configuration information and should prevent unauthorized or improper access where applicable.

10.6 Maintainability

The project should have a clear structure so that developers can easily understand, modify, update, and maintain the system.

10.7 Availability

The API should be available whenever the required weather information is requested, subject to the availability of the connected weather data service.

10.8 Compatibility

The API should be capable of communicating with different client applications through standard API request and response formats.

11. MODULES OF THE PROJECT:

The AI WeatherWise API is divided into different modules. Each module performs a specific function and together they provide the complete weather information service.

11.1 User / Client Module

This module allows the user or client application to request weather information. The user provides the required location or weather-related input.

11.2 API Request Module

This module receives the weather request from the client and passes the request to the appropriate processing component.

11.3 Request Processing Module

This module processes the received request and checks the required input before retrieving the weather information.

11.4 Weather Data Retrieval Module

This module obtains the required weather information from the configured weather data source or service.

11.5 Response Processing Module

This module processes the retrieved weather information and organizes it into a structured format.

11.6 API Response Module

This module sends the processed weather information back to the client application in a readable and structured format, such as JSON.

11.7 Error Handling Module

This module handles invalid requests, missing information, or other errors and provides an appropriate response to the user or client application.

11.8 Integration Module

This module allows the API to be integrated with other applications, websites, or services that require weather information.

12. API REQUEST AND RESPONSE:

The AI WeatherWise API communicates with a client application through API requests and responses. The client sends a request to obtain weather information, and the API processes the request and returns the required information in a structured format.

12.1 API Request

An API request is a request sent by the user or client application to obtain weather information. The request generally contains the location or other required parameters.

Example Request

Weather information request for: Chennai

The API receives the request and processes the provided location to obtain the required weather information.

12.2 Request Processing

After receiving the request, the system:

1. Receives the request from the client.
2. Checks the required input.
3. Processes the location information.
4. Retrieves the required weather data.
5. Prepares the response.

12.3 API Response

An API response is the information returned by the system after processing the request. The response is provided in a structured format so that it can be easily used by applications.

Example Response

{

"location": "Chennai",

"temperature": "30°C",

"condition": "Partly Cloudy",

"humidity": "70%"

}

The above response is an example of how weather information can be represented. The actual response depends on the data received from the configured weather service.

12.4 Response Information

The response may contain important weather details such as:

- Location
- Temperature

- Weather condition
- Humidity
- Wind speed or related information, where available

12.5 Error Response

If the request contains invalid or incomplete information, the API should return an appropriate error message instead of providing incorrect weather information.

Example:

```
{
  "error": "Invalid location or request"
}
```

Thus, the API request and response mechanism allows the AI WeatherWise API to communicate effectively with users and other applications.

13. IMPLEMENTATION:

The implementation of the AI WeatherWise API involves developing the API, processing weather requests, retrieving weather information, and returning the required data in a structured format. The project is implemented as a simple and usable API-based weather information system.

13.1 Project Setup

The required development environment and project files are prepared before starting the implementation. The project structure is organized so that the API components can be developed and maintained easily.

13.2 API Configuration

The weather service or data source required for obtaining weather information is configured with the project. The necessary API configuration is used to communicate with the weather data service.

13.3 Request Handling

The API is designed to receive weather requests from a client application. The requested location and other required parameters are processed by the system.

13.4 Weather Data Retrieval

After receiving a valid request, the system communicates with the configured weather data source and retrieves the required weather information.

13.5 Data Processing

The retrieved weather information is processed and organized into a structured format. Important information such as location, temperature, weather condition, and humidity can be included in the response.

13.6 Response Generation

After processing the weather data, the API generates a structured response. JSON format can be used to make the response easy for applications to read and process.

13.7 Error Handling

The system handles invalid or incomplete requests by generating suitable error responses. This helps users understand when the requested information cannot be processed.

13.8 API Usage

The completed API can be accessed by a suitable client application or API testing tool. A request is submitted with the required information, and the resulting weather data is displayed as the API response.

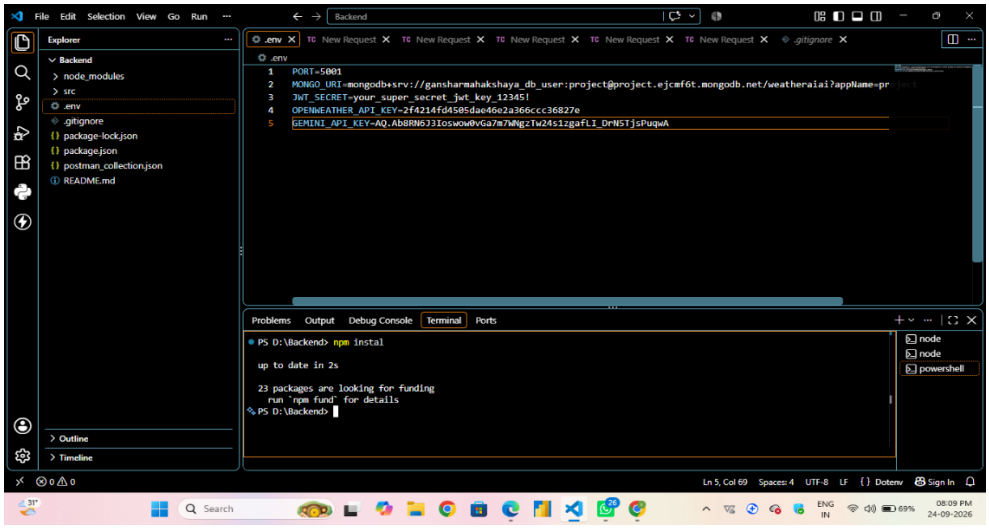
13.9 Implementation Summary

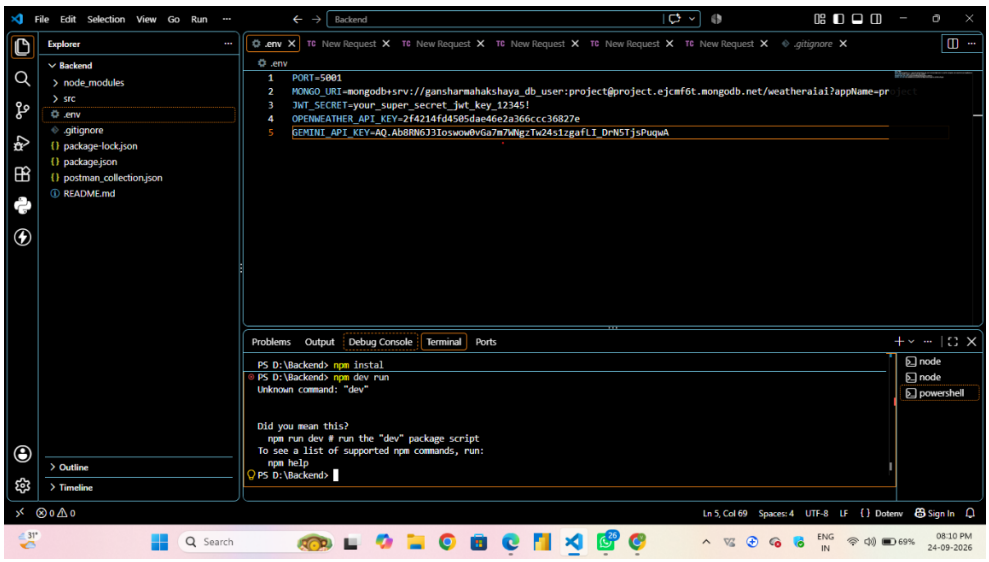
The implementation follows the overall flow:

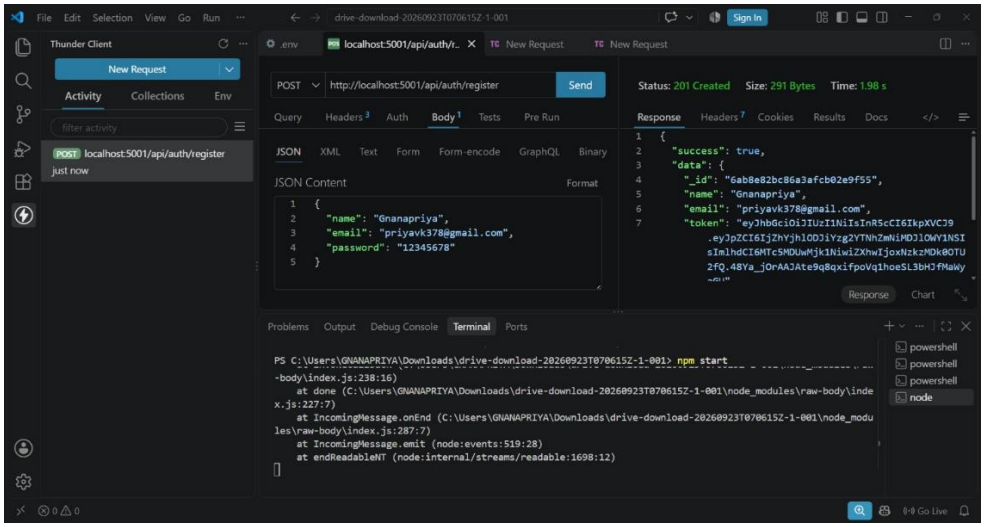
Client Request → Request Processing → Weather Data Retrieval → Data Processing → API Response

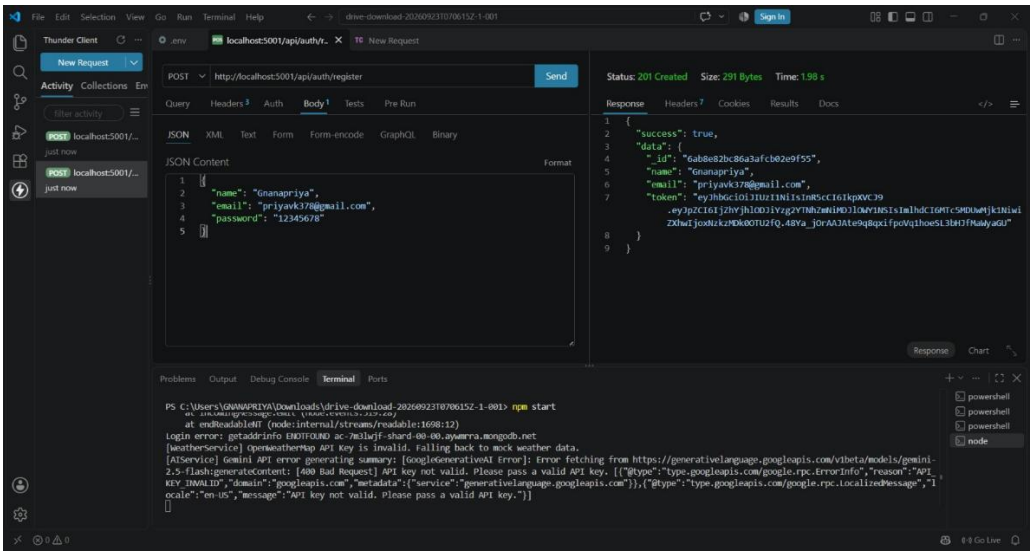
This implementation provides a simple way to obtain and use weather information through an API-based system.

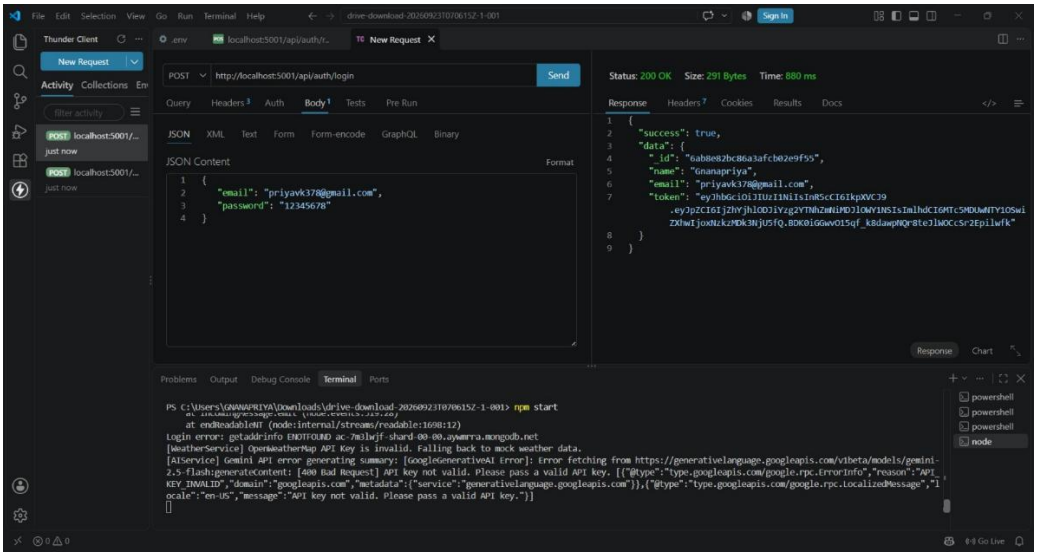
14. Screenshot:

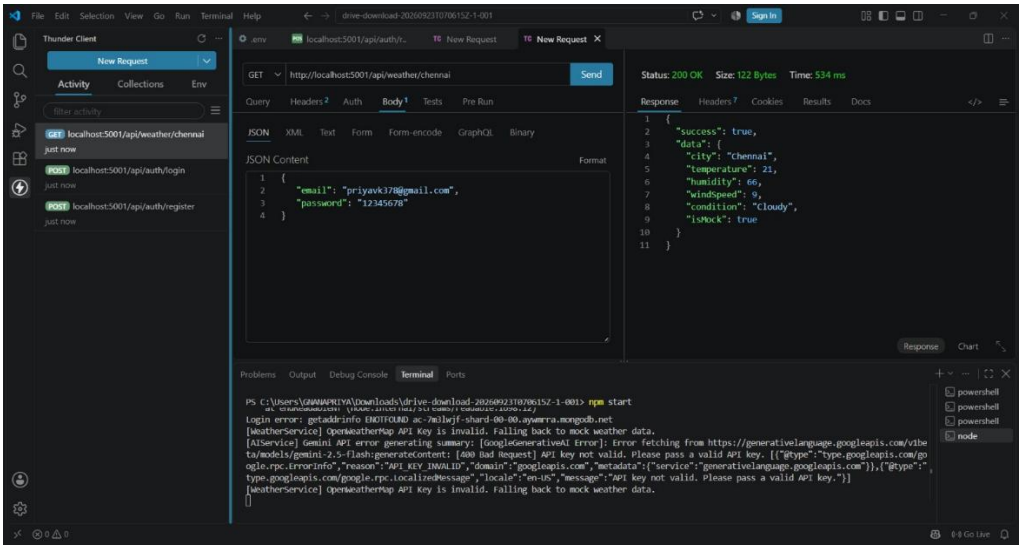


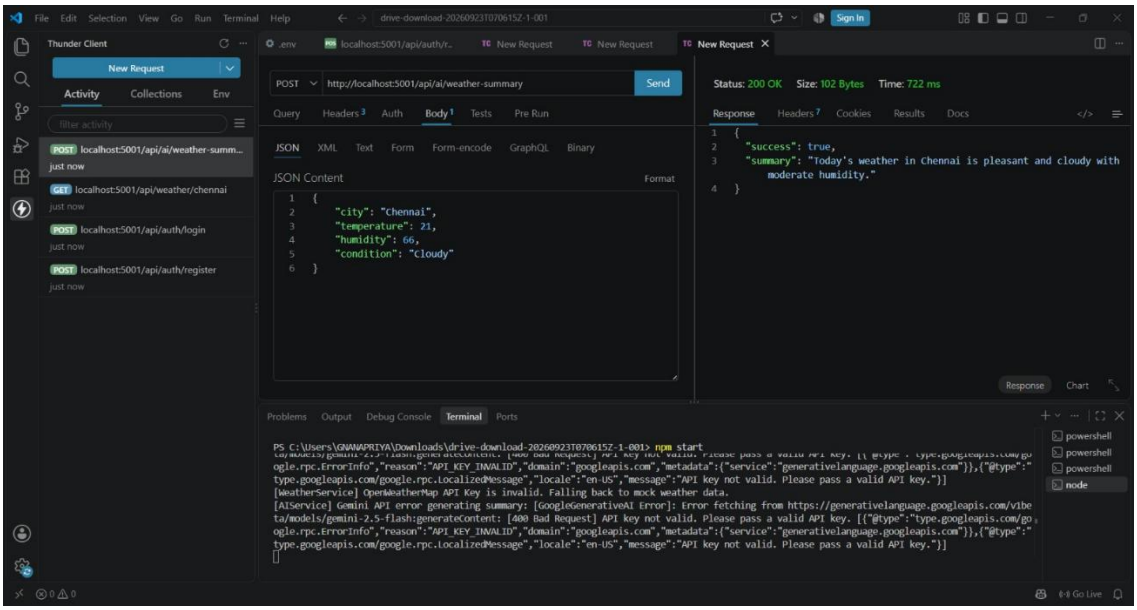


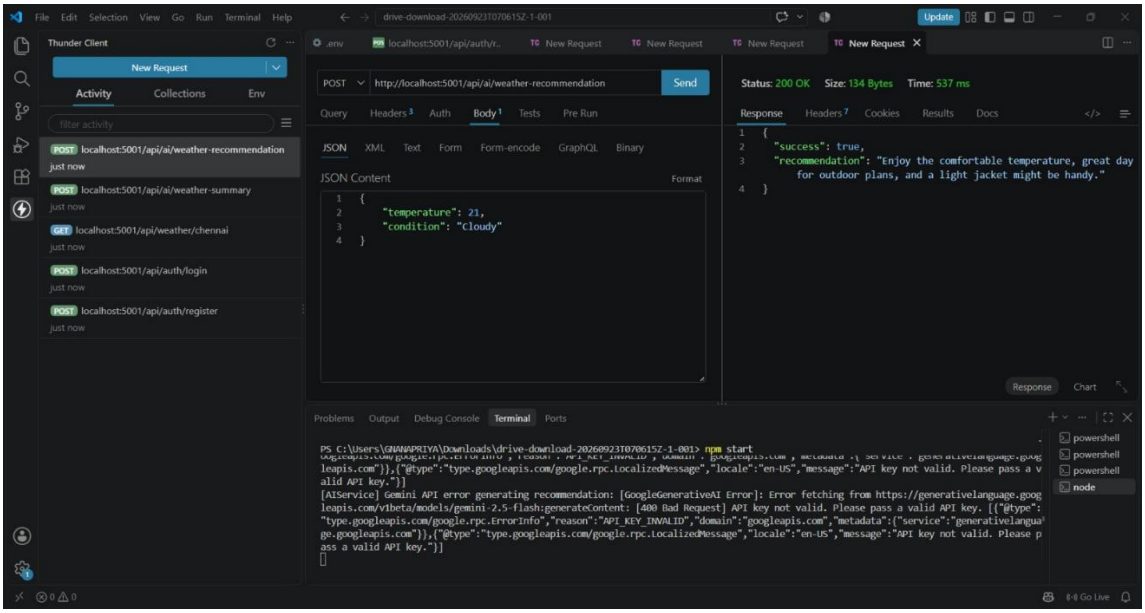

Thunder Client
New Request
Activity
Collections
Env
POST localhost:5001/api/auth/register
just now
POST http://localhost:5001/api/auth/register
Send
Status: 201 Created
Size: 291 Bytes
Time: 1.98 s
JSON Content
{
"name": "Gnanapriya",
"email": "priyavk378@gmail.com",
"password": "12345678"
}
"success": true,
"data": {
"_id": "6ab8e82bc86a3afcb02e9f55",
"name": "Gnanapriya",
"email": "priyavk378@gmail.com",
PS C:\Users\GNANAPRIYA\Downloads\drive-download-20260923T070615Z-1-001> npm start
-body\index.js:238:16)
at done (C:\Users\GNANAPRIYA\Downloads\drive-download-20260923T070615Z-1-001\node_modules\raw-body\index.js:227:7)
at IncomingMessage.onEnd (C:\Users\GNANAPRIYA\Downloads\drive-download-20260923T070615Z-1-001\node_modules\raw-body\index.js:287:7)
at IncomingMessage.emit (node:events:519:28)
at endReadableNT (node:internal/streams/readable:1698:12)


Thunder Client
New Request
POST http://localhost:5001/api/auth/register
Send
Status: 201 Created
Size: 291 Bytes
Time: 1.98 s
JSON Content
"success": true,
"data": {
"_id": "6ab8e82bc86a3afcb02e9f55",
"name": "Gnanapriya",
"email": "priyavk378@gmail.com",
PS C:\Users\GNANAPRIYA\Downloads\drive-download-20260923T070615Z-1-001> npm start
at endReadableNT (node:internal/streams/readable:1698:12)
Login error: getaddrinfo ENOTFOUND ac-7m3lwjf-shard-00-00.aywmrra.mongodb.net
[WeatherService] OpenWeatherMap API Key is invalid. Falling back to mock weather data.
[AIService] Gemini API error generating summary: [GoogleGenerativeAI Error]: Error fetching from https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent: [400 Bad Request] API key not valid. Please pass a valid API key. [{"@type":"type.googleapis.com/google.rpc.ErrorInfo","reason":"API_KEY_INVALID","domain":"googleapis.com","metadata":{"service":"generativelanguage.googleapis.com"}},{"@type":"type.googleapis.com/google.rpc.LocalizedMessage","locale":"en-US","message":"API key not valid. Please pass a valid API key."}]


Thunder Client
New Request
POST http://localhost:5001/api/auth/login
Send
Status: 200 OK Size: 291 Bytes Time: 880 ms
{
"email": "priyavk378@gmail.com",
"password": "12345678"
}
{
"success": true,
"data": {
"_id": "6ab8e82bc86a3afcb02e9f55",
"name": "Gnanapriya",
"email": "priyavk378@gmail.com",
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
.eyJpZCI6IjZhY jhlODJiYzg2YTNhZmNiMDJlOWY1NSIsImlhdCI6MTc5MDUwNTY1OSwi
ZXhwIjoxNzkzMDk3NjU5fQ.BDK0iGGwvO15qf_k8dawpNQr8te3lWOCcSr2Epilwfk"
}
}
PS C:\Users\GNANAPRIYA\Downloads\drive-download-20260923T070615Z-1-001> npm start
Login error: getaddrinfo ENOTFOUND ac-7m3lwjf-shard-00-00.aywmrra.mongodb.net
[WeatherService] OpenWeatherMap API Key is invalid. Falling back to mock weather data.
[AIService] Gemini API error generating summary: [GoogleGenerativeAI Error]: Error fetching from https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent: [400 Bad Request] API key not valid. Please pass a valid API key. [{"@type":"type.googleapis.com/google.rpc.ErrorInfo","reason":"API_KEY_INVALID","domain":"googleapis.com","metadata":{"service":"generativelanguage.googleapis.com"}},{"@type":"type.googleapis.com/google.rpc.LocalizedMessage","locale":"en-US","message":"API key not valid. Please pass a valid API key."}]


Thunder Client
New Request
GET http://localhost:5001/api/weather/chennai
Send
Status: 200 OK Size: 122 Bytes Time: 534 ms
{
"success": true,
"data": {
"city": "chennai",
"temperature": 21,
"humidity": 66,
"windSpeed": 9,
"condition": "Cloudy",
"isMock": true
}
}
{
"email": "priyavk378@gmail.com",
"password": "12345678"
}
PS C:\Users\GNANAPRIYA\Downloads\drive-download-20260923T070615Z-1-001> npm start
Login error: getaddrinfo ENOTFOUND ac-7m3lwjf-shard-00-00.aywmrra.mongodb.net
[WeatherService] OpenWeatherMap API Key is invalid. Falling back to mock weather data.
[AIService] Gemini API error generating summary: [GoogleGenerativeAI Error]: Error fetching from https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent: [400 Bad Request] API key not valid. Please pass a valid API key. [{"@type":"type.googleapis.com/google.rpc.ErrorInfo","reason":"API_KEY_INVALID","domain":"googleapis.com","metadata":{"service":"generativelanguage.googleapis.com"}},{"@type":"type.googleapis.com/google.rpc.LocalizedMessage","locale":"en-US","message":"API key not valid. Please pass a valid API key."}]
[WeatherService] OpenWeatherMap API Key is invalid. Falling back to mock weather data.


Thunder Client
New Request
POST http://localhost:5001/api/ai/weather-summary
Send
Status: 200 OK Size: 102 Bytes Time: 722 ms
{
"success": true,
"summary": "Today's weather in Chennai is pleasant and cloudy with moderate humidity."
}
{
"city": "Chennai",
"temperature": 21,
"humidity": 66,
"condition": "Cloudy"
}
PS C:\Users\GNANAPRIYA\Downloads\drive-download-20260923T070615Z-1-001> npm start
[WeatherService] OpenWeatherMap API Key is invalid. Falling back to mock weather data.
[AIService] Gemini API error generating summary: [GoogleGenerativeAI Error]: Error fetching from https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent: [400 Bad Request] API key not valid. Please pass a valid API key. [{"@type":"type.googleapis.com/google.rpc.ErrorInfo","reason":"API_KEY_INVALID","domain":"googleapis.com","metadata":{"service":"generativelanguage.googleapis.com"}},{"@type":"type.googleapis.com/google.rpc.LocalizedMessage","locale":"en-US","message":"API key not valid. Please pass a valid API key."}]



15. ADVANTAGES:

The AI WeatherWise API provides a simple and efficient way to access weather information through an API-based system. The major advantages of the project are given below.

15.1 Easy Access to Weather Information

Users can obtain weather information by providing the required location or parameters.

15.2 Simple API Communication

The system uses API requests and responses to communicate between the client and the weather service.

15.3 Structured Data

Weather information can be provided in a structured format such as JSON, making it easy for applications to read and process.

15.4 Easy Integration

The API can be integrated with websites, mobile applications, and other software systems that require weather information.

15.5 Saves Development Time

Applications can use the available weather API instead of developing a complete weather data system from the beginning.

15.6 User-Friendly Information

Important weather details such as temperature, condition, and humidity can be presented in a clear and understandable format.

15.7 Flexible Usage

The API-based design allows the weather information service to be used by different types of client applications.

15.8 Easy to Maintain

The modular structure of the project makes it easier to modify, update, and maintain individual components.

16. LIMITATIONS:

Although the AI WeatherWise API provides a useful way to access weather information, the project has certain limitations. These limitations may depend on the API configuration, weather data source, and internet connectivity.

16.1 Internet Dependency

The system generally requires an active internet connection to communicate with the weather data service and obtain updated information.

16.2 Dependency on Weather Data Source

The accuracy and availability of weather information depend on the configured weather data source or service.

16.3 API Service Availability

If the connected weather service is temporarily unavailable, the application may not be able to retrieve the required weather information.

16.4 API Usage Restrictions

The weather data service may have limitations such as request limits, usage restrictions, or other service conditions.

16.5 Data Accuracy

Weather information may not always be completely accurate because weather conditions can change frequently and the system depends on the data provided by the external service.

16.6 Limited Information

The information available through the project depends on the weather data provided by the configured service. Some advanced weather details may not be available.

16.7 Configuration Dependency

Changes to the external weather service or API configuration may require corresponding updates to the project.

16.8 Error Handling

Unexpected network problems, invalid requests, or external service errors may affect the availability of weather information.

17. FUTURE ENHANCEMENT:

The AI WeatherWise API can be further improved by adding new features and technologies. Future enhancements can make the system more useful, intelligent, and suitable for a wider range of applications.

17.1 Weather Forecasting

The system can be enhanced to provide weather forecasts for upcoming days instead of displaying only current weather information.

17.2 AI-Based Weather Analysis

Artificial Intelligence and Machine Learning techniques can be added to analyse weather data and provide more meaningful weather insights.

17.3 Multiple Location Support

The system can be enhanced to allow users to compare weather information from multiple locations.

17.4 Weather Alerts

The project can include notifications or alerts for severe weather conditions such as heavy rain, strong winds, or extreme temperatures.

17.5 Voice-Based Interaction

Voice input can be added so that users can request weather information using voice commands.

17.6 Mobile Application Integration

The API can be integrated with a mobile application to provide weather information through smartphones.

17.7 Advanced Weather Information

Additional information such as air quality, UV index, sunrise and sunset times, and extended forecasts can be included when supported by the data source.

17.8 Improved User Interface

A more interactive and user-friendly interface can be developed to display weather information using charts, icons, and visual indicators.

17.9 Cloud Deployment

The project can be deployed on a cloud platform to improve accessibility and support applications from different locations.

17.10 Real-Time Updates

The system can be enhanced to provide regularly updated weather information whenever new data is available.

18. APPLICATIONS:

The AI WeatherWise API can be used in different applications and systems that require weather-related information. Some of its possible applications are given below.

18.1 Weather Applications

The API can be used in weather applications to retrieve and display current weather information.

18.2 Websites

Websites can integrate the API to display weather information based on the user's selected location.

18.3 Mobile Applications

Mobile applications can use the API to provide weather information to users on smartphones and other mobile devices.

18.4 Travel Applications

Travel applications can use weather information to help users understand weather conditions at their destination.

18.5 Agriculture Applications

Weather information can support agricultural applications by providing useful information about temperature, humidity, rainfall, and other available weather conditions.

18.6 Educational Projects

The API can be used as a learning project for students to understand APIs, JSON data, web services, and application integration.

18.7 Smart Applications

The system can be integrated into smart applications that require weather information for decision-making or automated services.

18.8 Location-Based Services

Applications that provide location-based services can use the API to display weather information for a selected location.

18.9 Event Planning Applications

Weather information can be integrated into event planning systems to help users consider weather conditions while planning outdoor activities.

18.10 Other Software Systems

Any software application that requires weather information can potentially integrate the AI WeatherWise API, depending on its requirements and the available weather data.

19. CONCLUSION:

The AI WeatherWise API is an API-based project developed to provide weather information in a simple and structured manner. The system receives weather-related requests, processes the required information, retrieves data from the configured weather service, and provides the result to the client application.

The project demonstrates the practical use of API communication, structured data, request processing, and application integration. It provides a foundation for developing applications that require weather information.

The project can also be enhanced in the future by adding features such as AI-based weather analysis, weather forecasting, alerts, voice interaction, mobile application integration, and real-time updates.

Overall, the AI WeatherWise API provides a useful and flexible approach for accessing and integrating weather information into different applications.

GITHUB LINK:

DRIVE LINK:

https://drive.google.com/file/d/142JyMHDVoddszT8nu8C789IeYliylyAH/view?usp=drivesdk